

Submission Worksheet

Submission Data

Course: IT202-008-S2026

Assignment: IT202 Milestone 1 - 2026

Student: George C. (gec23)

Status: Submitted | **Worksheet Progress:** 80%

Potential Grade: 9.00/10.00 (90.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 4/12/2026 2:16:47 PM

Updated: 4/13/2026 11:46:16 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone1-2026/gec23>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-1-2026/view/gec23>

Instructions

- Overview Link: <https://youtu.be/IDtqdaLwcVg>

1. Refer to Milestone1 of this doc:

<https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view>

2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone1 branch
 1. `git checkout Milestone1` (ensure proper starting branch)
 2. `git pull origin Milestone1` (ensure history is up to date)
5. Fill out the below worksheet
 - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
 - Ensure proper styling is applied to each page
 - Ensure there are no visible technical errors; only user-friendly messages are allowed
6. Once finished, click "Submit and Export"
7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 1. `git add .`
 2. `git commit -m "adding PDF"`
 3. `git push origin Milestone1`
 4. On Github merge the pull request from Milestone1 to qa
 5. On Github create a pull request from qa to prod and immediately merge. (This will trigger the prod deploy to make the heroku prod links work)
8. Upload the same PDF to Canvas
9. Sync Local
10. `git checkout qa`
11. `git pull origin qa`

Section #1: (2 pts.) Feature: User Will Be Able To Register A New Account

Task #1 (0.67 pts.) - Validation

Progress: 50%

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

- Show the code related to the register page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)



code

Register

Enter an email address

First Name

Last Name

Password

Confirm

Register

add email

[login](#) [register](#)

[login](#) [register](#)

Register

password

[login](#) [register](#)

Invalid email address.
Passwords must match.

invalid email + password

 Saved: 4/12/2026 6:18:03 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

the validation step checks the inputs of the user and displays a message if anything goes wrong during the process, making sure that everything is done correctly before continuing

 Saved: 4/12/2026 6:18:03 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 50%

Part 1:

Progress: 100%

Details:

- Show the code related to the register page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm](you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

code



Saved: 4/13/2026 11:04:42 PM

≡ Part 2:

Progress: 0%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

Missing Response



Saved: 4/13/2026 11:04:42 PM

≡ Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 0%



Part 1:

Progress: 0%

Details:

- Show the code related to the register page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit

- the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the `validate()` function or use the provided helper script in the instructions
- Include the outputs related to username/email already in use



Missing Caption



Saved: 4/1/2026 9:45:42 PM

≡ Part 2:

Progress: 0%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

Missing Response



Saved: 4/1/2026 9:45:42 PM

↔ Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , , zero credit if it does not)
- Include the render.com prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

<https://github.com/gcruzmedina/gec23->



URL

<https://github.com/gcruzmec>



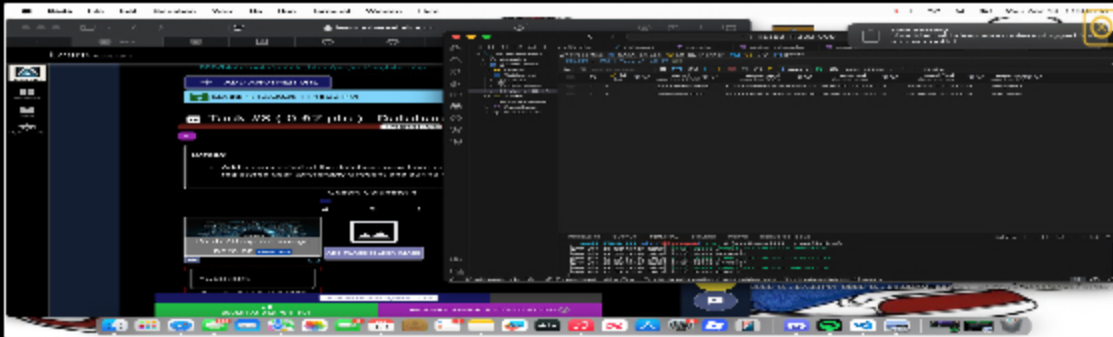
Saved: 4/13/2026 11:45:33 PM

Task #3 (0.67 pts.) - Database - Users table

Progress: 100%

Details:

- Add a screenshot of the database view from vs code showing a valid registered user (preferably a recent one during evidence capturing)



users



Saved: 4/13/2026 11:46:16 PM

Section #2: (2 pts.) Feature: User Will Be Able To Login To Their Account

Progress: 66%

Task #1 (0.67 pts.) - Validation

Progress: 0%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 0%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for

the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 0%

Details:

- Show the code related to the login page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)



Missing Caption



Saved: 4/1/2026 9:45:42 PM

Part 2:

Progress: 0%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

Missing Response



Saved: 4/1/2026 9:45:42 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 0%

Part 1:

Progress: 0%

Details:

- Show the code related to the login page's validate() function

- Show examples of each validation message [username format, email format, password format] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag



Missing Caption



Saved: 4/1/2026 9:45:42 PM

≡ Part 2:

Progress: 0%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

Missing Response



Saved: 4/1/2026 9:45:42 PM

≡ Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 0%

📄 Part 1:

Progress: 0%

Details:

- Show the code related to the login page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the `validate()` function or use the provided helper script in the instructions

- Include the outputs related to user doesn't exist and php-side password mismatch (the one that checks against the DB hash)



Missing Caption



Saved: 4/1/2026 9:45:42 PM

Part 2:

Progress: 0%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

Missing Response



Saved: 4/1/2026 9:45:42 PM

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , , zero credit if it does not)
- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

[https://github.com/gcruzmedina/gec23-](https://github.com/gcruzmedina/gec23-IT2023008/main/public_html/project/profile.php)



URL

[https://github.com/gcruzmed](https://github.com/gcruzmedina/gec23-IT2023008/main/public_html/project/profile.php)



[IT2023008/main/public_html/project/profile.php](https://github.com/gcruzmedina/gec23-IT2023008/main/public_html/project/profile.php)

URL #2

[https://github.com/gcruzmedina/gec23-](https://github.com/gcruzmedina/gec23-IT2023008/main/public_html/project/profile.php)



URL

[https://github.com/gcruzmed](https://github.com/gcruzmedina/gec23-IT2023008/main/public_html/project/profile.php)



IT202-008/pull/5

URL #3

[https://gec23-](https://gec23-it202-008.onrender.com)

[it202-008.onrender.com/project/profile.php](https://gec23-it202-008.onrender.com/project/profile.php)



URL

<https://gec23-it202-008.onrender.com>



Saved: 4/13/2026 11:44:00 PM

Task #3 (0.67 pts.) - User Session Evidence

Progress: 100%

Details:

- From the render.com logs (Logs tab), capture the session err
- It should show the id, username, email, and roles



logs



Saved: 4/13/2026 11:40:20 PM

Section #3: (1 pt.) Feature: User Will Be Able To Logout

Progress: 100%

Task #1 (1 pt.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the successful logout message (visit the proper file on Render.com qa after a manual deploy)
- Show the code snippet of the logout page

[Login](#) [Register](#)

You have been logged out

[Login](#)

 Saved: 4/13/2026 11:35:31 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1 

URL

 Saved: 4/13/2026 11:35:31 PM

Part 3:

Progress: 100%

Details:

- Briefly explain how the logout process works and how the user is officially logged off

Your Response:

the logout function basically wipes the memory and deletes the user history

 Saved: 4/13/2026 11:35:31 PM

Section #4: (2 pts.) Feature: Security Rules

Progress: 100%

Task #1 (1 pt.) - Database Tables

Progress: 100%

- From the vs code mysql tool, show both the `Roles` and `User`



≡ Task #2 (1 pt.) - Demo Security Rules

Progress: 100%

Part 1:

Progress: 100%

- Show the message related to needing to be logged in (i.e., try to
- Show the message related to not having the proper permission/role
- Include a snippet of the login check function
- Include a snippet of the role check function





profile

 Saved: 4/13/2026 11:43:29 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1

<https://github.com/gcruzmedina/gec23-IT20240081>



URL

<https://github.com/gcruzmedina/>

 Saved: 4/13/2026 11:43:29 PM

Part 3:

Progress: 100%

Details:

- Briefly explain how the login check code works
- Briefly explain how the role check code works

Your Response:

the code acts as a gatekeeper, which blocks the user from reaching something they are not supposed to without permission.

 Saved: 4/13/2026 11:43:29 PM

Section #5: (2 pts.) Feature: User Profile

Progress: 100%

Task #1 (1 pt.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

The screenshot shows a code editor with a file explorer on the left. The main editor displays HTML and JavaScript code. The HTML code includes a form with input fields for username, email, password, and confirm password. The JavaScript code defines validation functions for these fields, such as `validateUsername`, `validateEmail`, `validatePassword`, and `validateConfirmPassword`. These functions use regular expressions and conditional logic to check the validity of the user input. The code is well-commented and organized into sections.

code

The screenshot shows a web form titled "Login Data Login" on an orange background. The form contains several input fields and buttons. Above the form, there are two error messages in yellow boxes: "Username and password must match" and "Password and Confirm password must match". The form has buttons for "Login", "Register", "Forgot Password", "New Password", "Online Password", and "Data Reset". The "Login" button is highlighted in blue. The form is designed to be user-friendly and clear, with error messages providing immediate feedback to the user.

error

Saved: 4/13/2026 10:57:45 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

keeps the user in check by alerting them of what is going wrong



Saved: 4/13/2026 10:57:45 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

```
47 <script>
48   // Profile Validation
49   const validate = (e) => {
50     e.preventDefault();
51     const username = document.getElementById('username').value;
52     const email = document.getElementById('email').value;
53     const password = document.getElementById('password').value;
54     const confirmPassword = document.getElementById('confirm').value;
55     let isValid = true;
56     if (!username.match(/^[a-zA-Z0-9]{3,10}$/)) {
57       document.getElementById('username').style.borderColor = 'red';
58       document.getElementById('username').style.backgroundColor = 'pink';
59       document.getElementById('username').value = '';
60       isValid = false;
61     }
62     if (!email.match(/^[a-zA-Z0-9]{3,10}@gmail.com$/)) {
63       document.getElementById('email').style.borderColor = 'red';
64       document.getElementById('email').style.backgroundColor = 'pink';
65       document.getElementById('email').value = '';
66       isValid = false;
67     }
68     if (password.length < 8 || !password.match(/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*^&quot;])[0-9a-zA-Z@$!%*^&quot;]{8,}$/)) {
69       document.getElementById('password').style.borderColor = 'red';
70       document.getElementById('password').style.backgroundColor = 'pink';
71       document.getElementById('password').value = '';
72       isValid = false;
73     }
74     if (password !== confirmPassword) {
75       document.getElementById('confirm').style.borderColor = 'red';
76       document.getElementById('confirm').style.backgroundColor = 'pink';
77       document.getElementById('confirm').value = '';
78       isValid = false;
79     }
80     if (isValid) {
81       document.getElementById('submit').disabled = false;
82     } else {
83       document.getElementById('submit').disabled = true;
84     }
85   };
86   document.getElementById('validate').addEventListener('click', validate);
87 </script>
```

code



Saved: 4/13/2026 10:32:15 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

like the other ones, it double checks to make sure that the correct information is being added in order to continue



Saved: 4/13/2026 10:32:15 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the `validate()` function or use the provided helper script in the instructions

```
$new_username = get_username(); // get_username() from database
// handle email / username updates
if (isset($_POST["email"]) || isset($_POST["username"])) {
    $new_email = $_POST["email"];
    $new_username = $_POST["username"];
    $hasError = false;
    // validate email
    if (empty($new_email)) {
        //echo "Email must not be empty<br>";
        flash("Email must not be empty.", "danger");
        $hasError = true;
    }
    // Sanitize and validate email
    $new_email = sanitize_email($new_email);
    if (!is_valid_email($new_email)) {
        //echo "Invalid email address<br>";
        flash("Invalid email address.", "danger");
        $hasError = true;
    }
    if (!is_valid_username($new_username)) {
        flash("Username must be lowercase, alphanumeric, and can only contain _", "danger");
        $hasError = true;
    }
}
```

email/user

```
// handle password update
if (isset($_POST["currentPassword"], $_POST["newPassword"], $_POST["confirmPassword"])) {
    //check/update password
    $current_password = $_POST["currentPassword"];
    $new_password = $_POST["newPassword"];
    $confirm_password = $_POST["confirmPassword"];
    // require all 3 to be set before attempting to process
    $can_update = !empty($current_password) && !empty($new_password) && !empty($confirm_password);
    if ($can_update) {
        // check that new matches confirm (i.e., no typos)
        if (!is_valid_confirm($new_password, $confirm_password)) {
            flash("New passwords don't match", "warning");
        } else {
            //validate current password against password rules
            $hasError = false;
            if (!is_valid_password($new_password)) {
                //echo "Password too short<br>";
                flash("Password must be at least 8 characters long.", "danger");
                $hasError = true;
            }
            if ($hasError) {
                // fetch current hash
            }
        }
    }
}
```

password

[Landing](#) [Profile](#) [Logout](#)

Invalid email address.

Profile

[Email](#)
[Username](#)
[Password Reset](#)
[Current Password](#)
[New Password](#)
[Confirm Password](#)

1st

[Landing](#) [Profile](#) [Logout](#)

Username must be lowercase, alphanumerical, and can only contain _ or -

Profile

[Email](#)
[Username](#)
[Password Reset](#)
[Current Password](#)
[New Password](#)
[Confirm Password](#)

2nd

[Landing](#) [Profile](#) [Logout](#)

Password and Confirm password must match.

Profile

[Email](#)
[Username](#)
[Password Reset](#)
[Current Password](#)
[New Password](#)
[Confirm Password](#)

3rd



Saved: 4/12/2026 8:06:30 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

the validation uses both php and js in order to perform security and also add to the user experience. php allows the function to go through information and to not have duplicates by being aware of what has already been submitted.



Saved: 4/12/2026 8:06:30 PM

Task #2 (1 pt.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , , zero credit if it does not)
- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

<https://gec23-it202-008.onrender.com/project/profile.php>



URL

<https://gec23-it202-008.onrender.com>



URL #2

https://github.com/gcruzmedina/gec23-IT202-008/Milestone1/public_html/project/profile.php



URL

<https://github.com/gcruzmedina/gec23-IT202-008/pull/20>



<https://github.com/gcruzmedina/gec23-IT202-008/pull/20>

URL #3

<https://github.com/gcruzmedina/gec23-IT202-008/pull/20>



URL

<https://github.com/gcruzmedina/gec23-IT202-008/pull/20>



Saved: 4/12/2026 8:10:53 PM

Section #6: (1 pt.) Misc

Progress: 100%

Task #1 (0.33 pts.) - Github Details

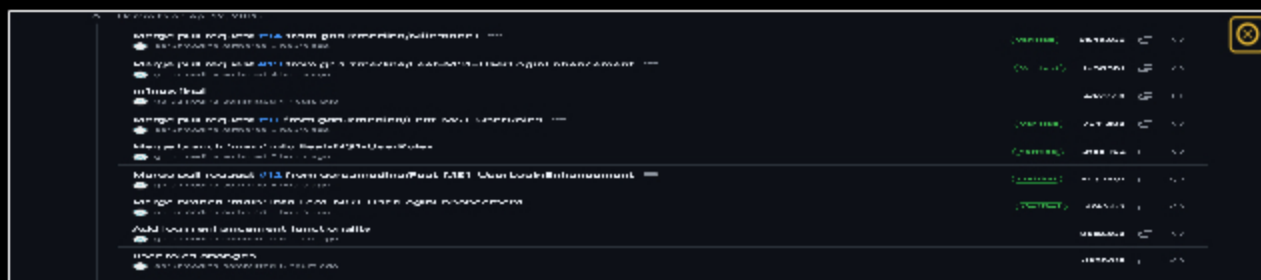
Progress: 100%

Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history (it may not be possible to capture all, just grab a reasonable chunk)



commits

 Saved: 4/12/2026 6:31:03 PM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request (should end in `/pull/#`) (Milestone1 into qa)

Include the link to the Pull Request (should end in `/pull/#`) (Milestone1 into qa)

URL #1
<https://github.com/gcruzmedina/gec23-IT20241020>



URL
<https://github.com/gcruzmedina/>

IT202400820

 Saved: 4/12/2026 6:31:03 PM

 Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

- Visit the [WakaTime.com Dashboard](#)
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

- Visit the [WakaTime.com Dashboard](#)
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary





ss2



ss3



Saved: 4/12/2026 2:23:00 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

i learned how to create a working website using localhost that involves a login page and other attachments which cause it to run correctly



Saved: 4/12/2026 2:17:18 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

following the assignments in order to get the product running. they were clear enough to understand without having to run into too many issues.



Saved: 4/12/2026 2:18:09 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

understanding how everything works correctly because all of this is very new and foreign to me. there was a lot of things i have never heard about before.



Saved: 4/12/2026 2:18:56 PM